

Lecture 2

VLSI Design Flow overview

Prof Peter YK Cheung

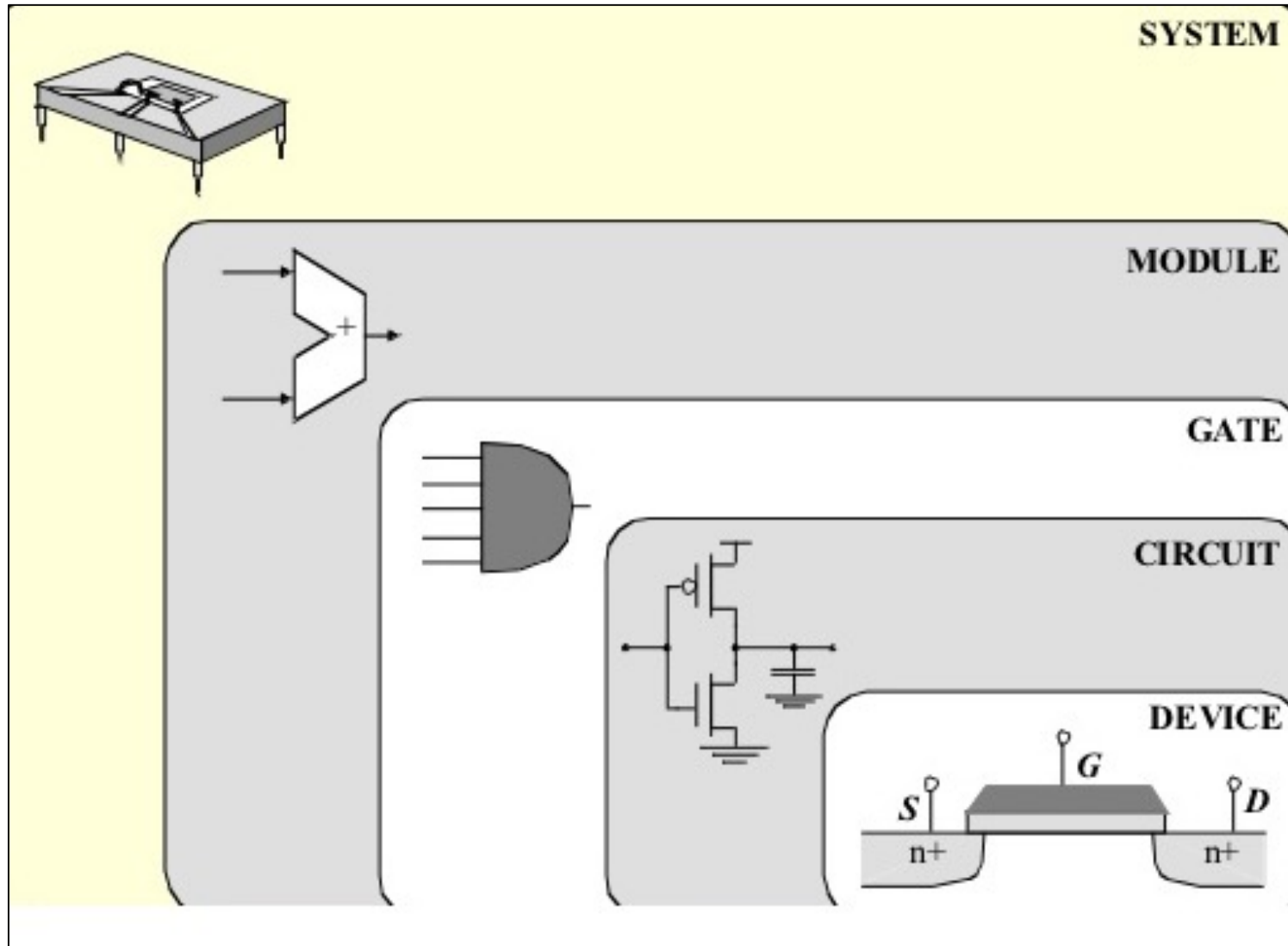
Department of Electrical & Electronic Engineering



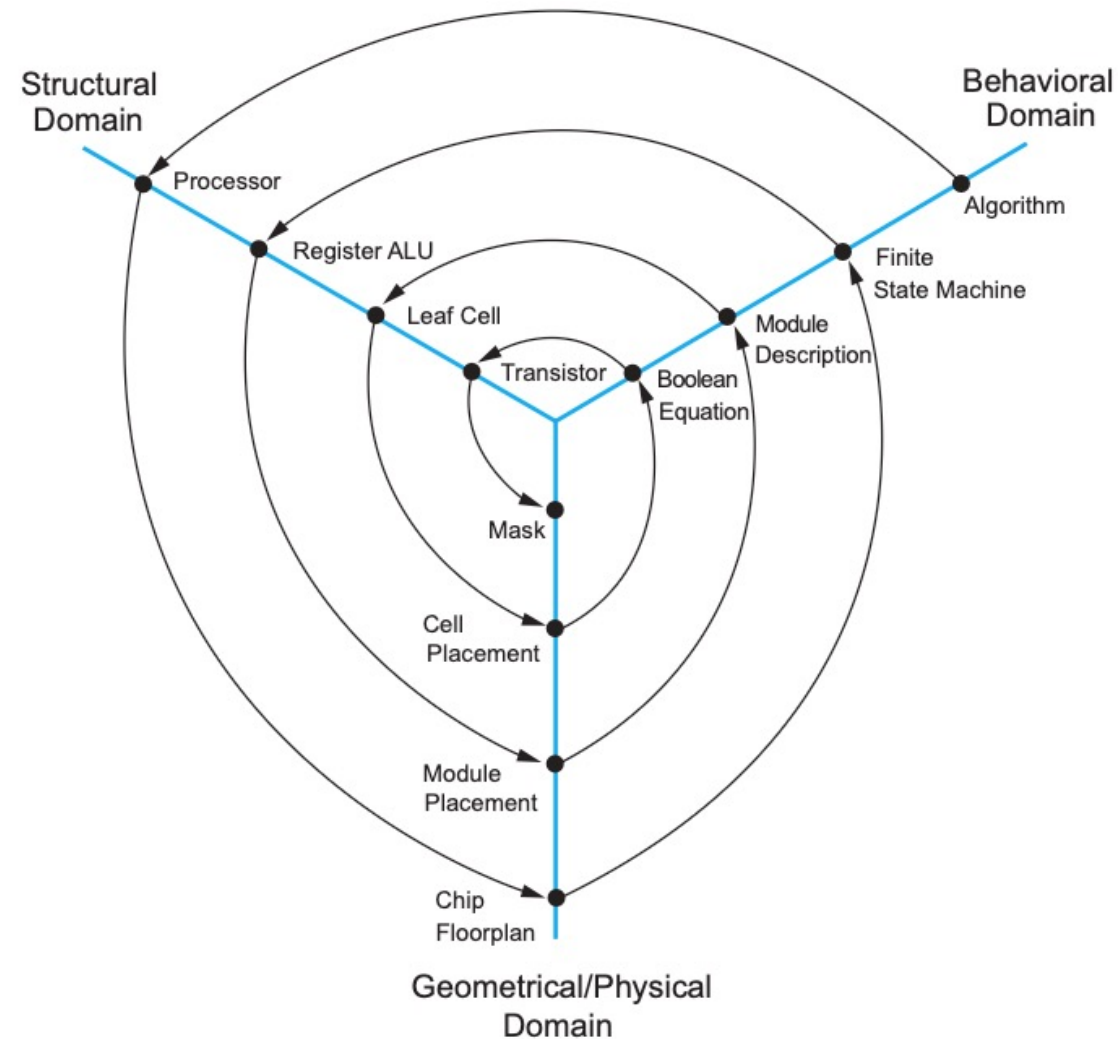
URL: <https://elec70142-digital-vlsi-design.github.io/course-website/>

E-mail: p.cheung@imperial.ac.uk

Levels of Abstraction in Digital Design

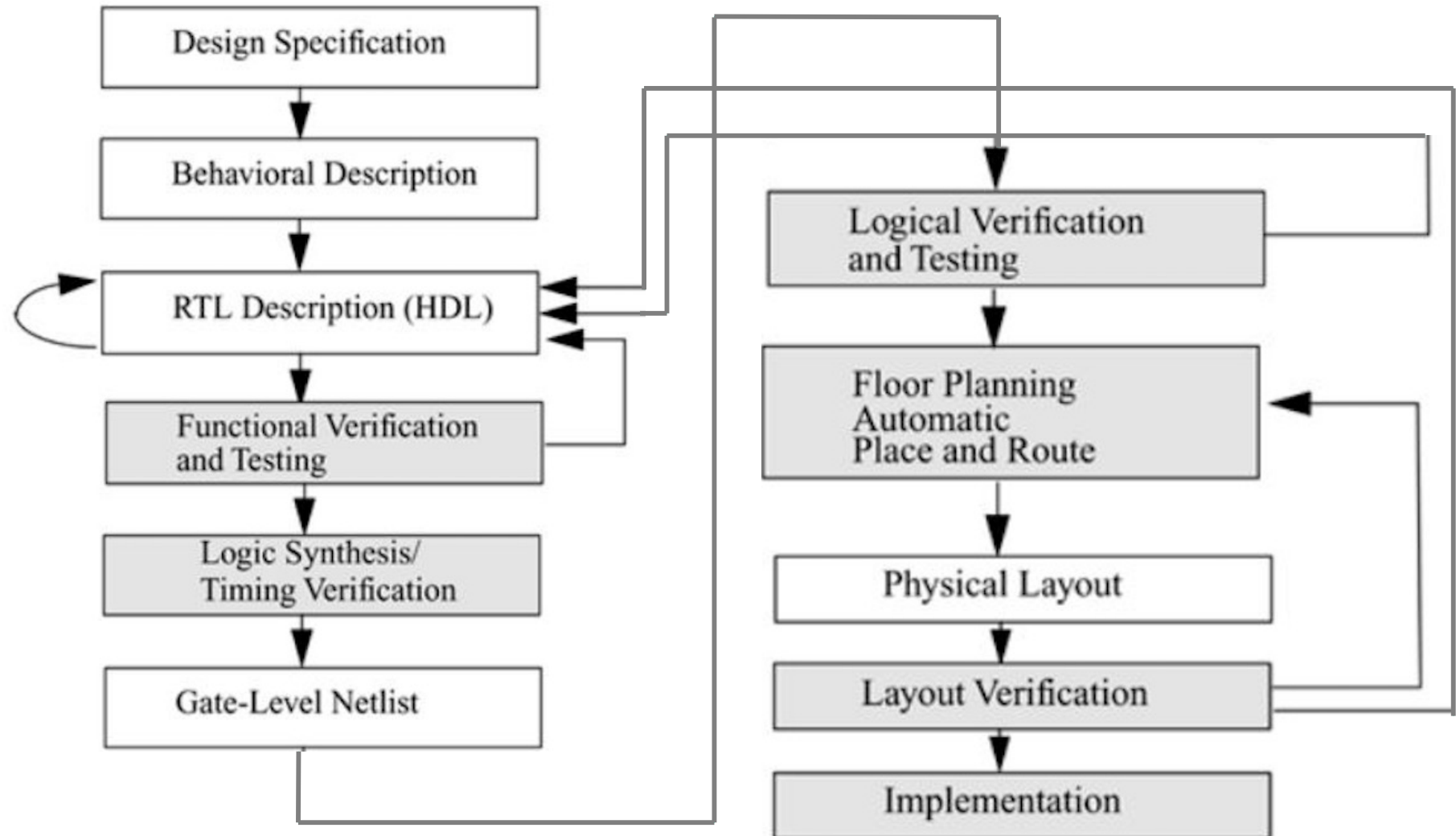


Levels of Abstraction in Digital Design

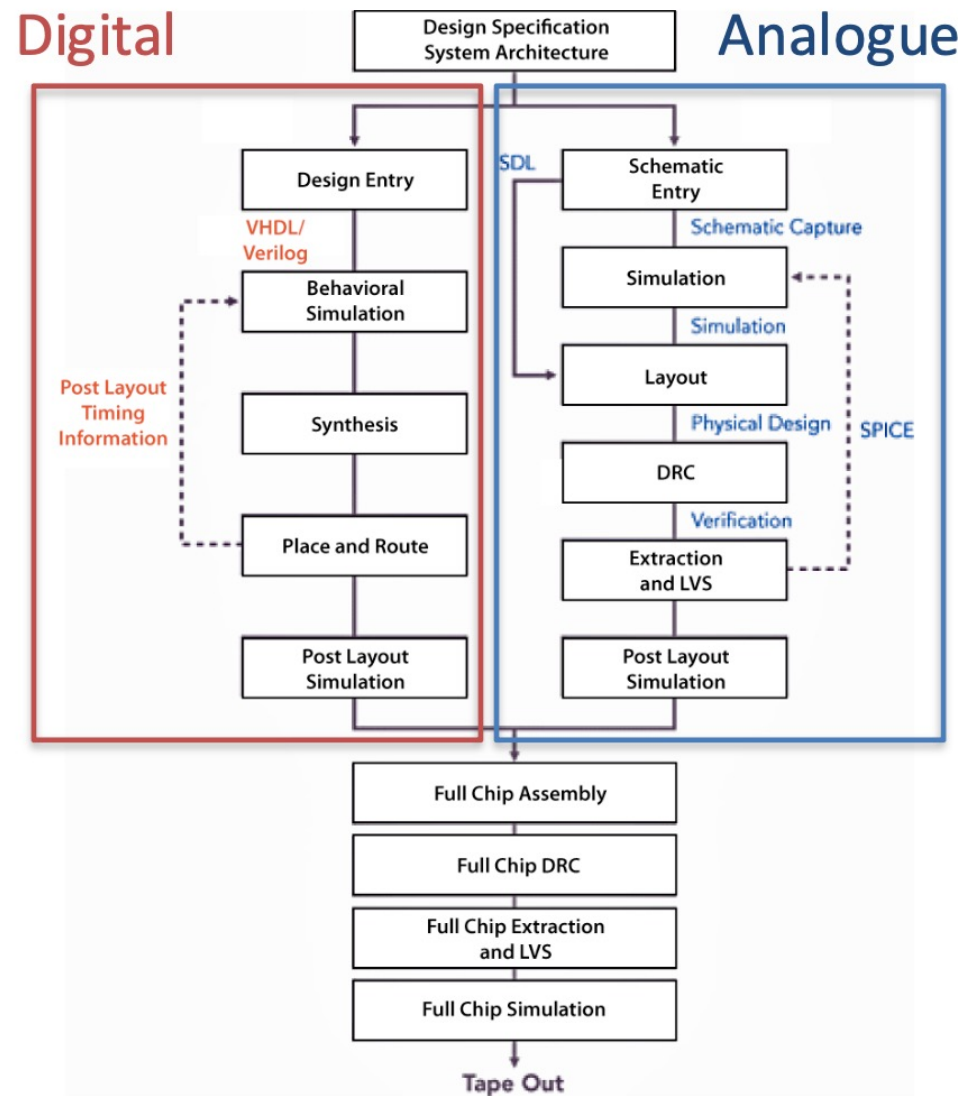


Source: Gajski 1983 & Kang 2003

Full-Custom VLSI Design Flow

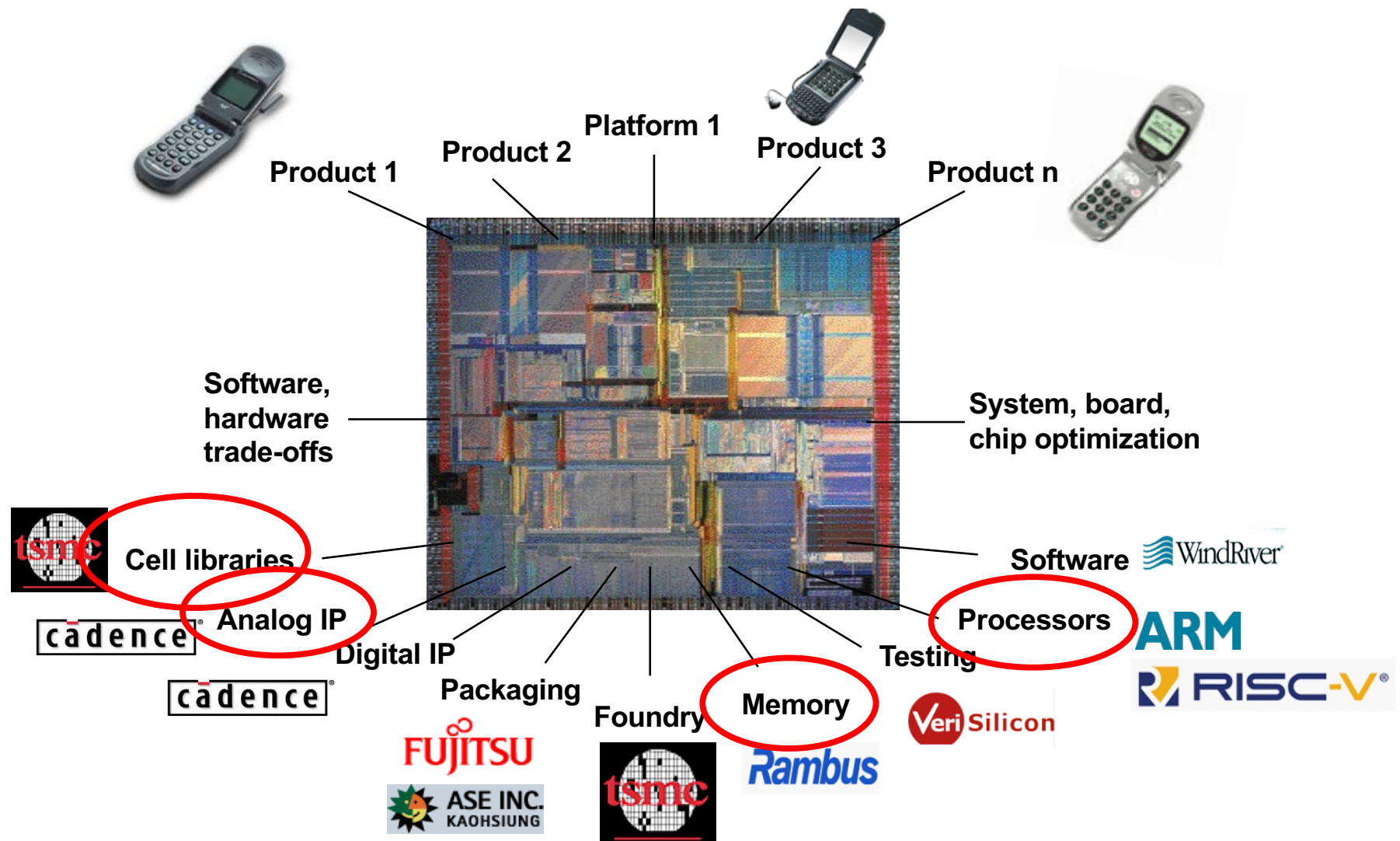


Mixed signal VLSI Design Flow



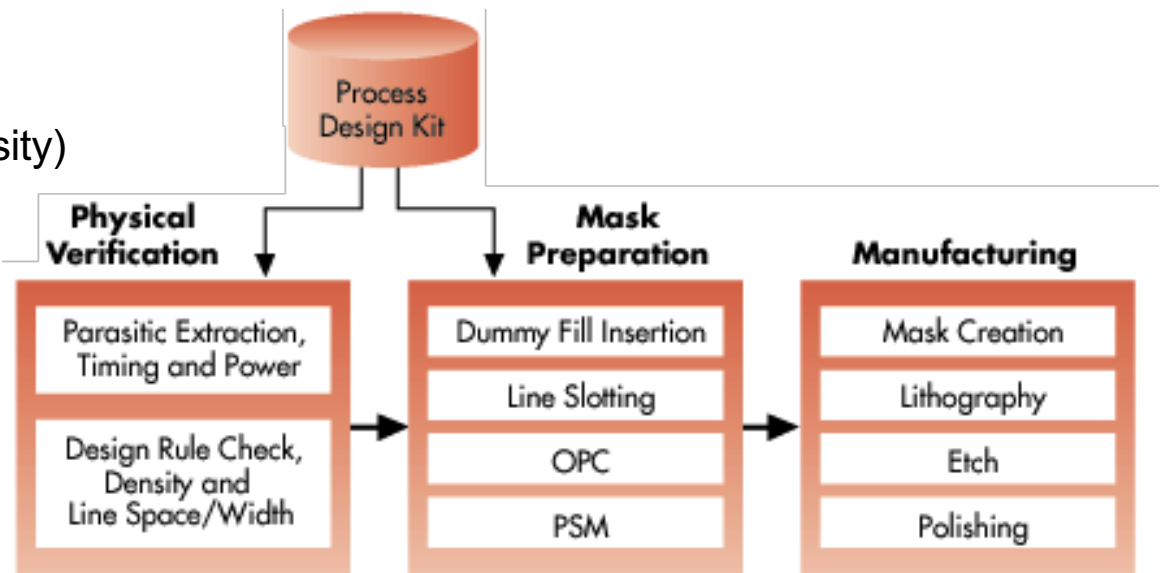
Source: Notes from T Constandinou

IP based VLSI Design

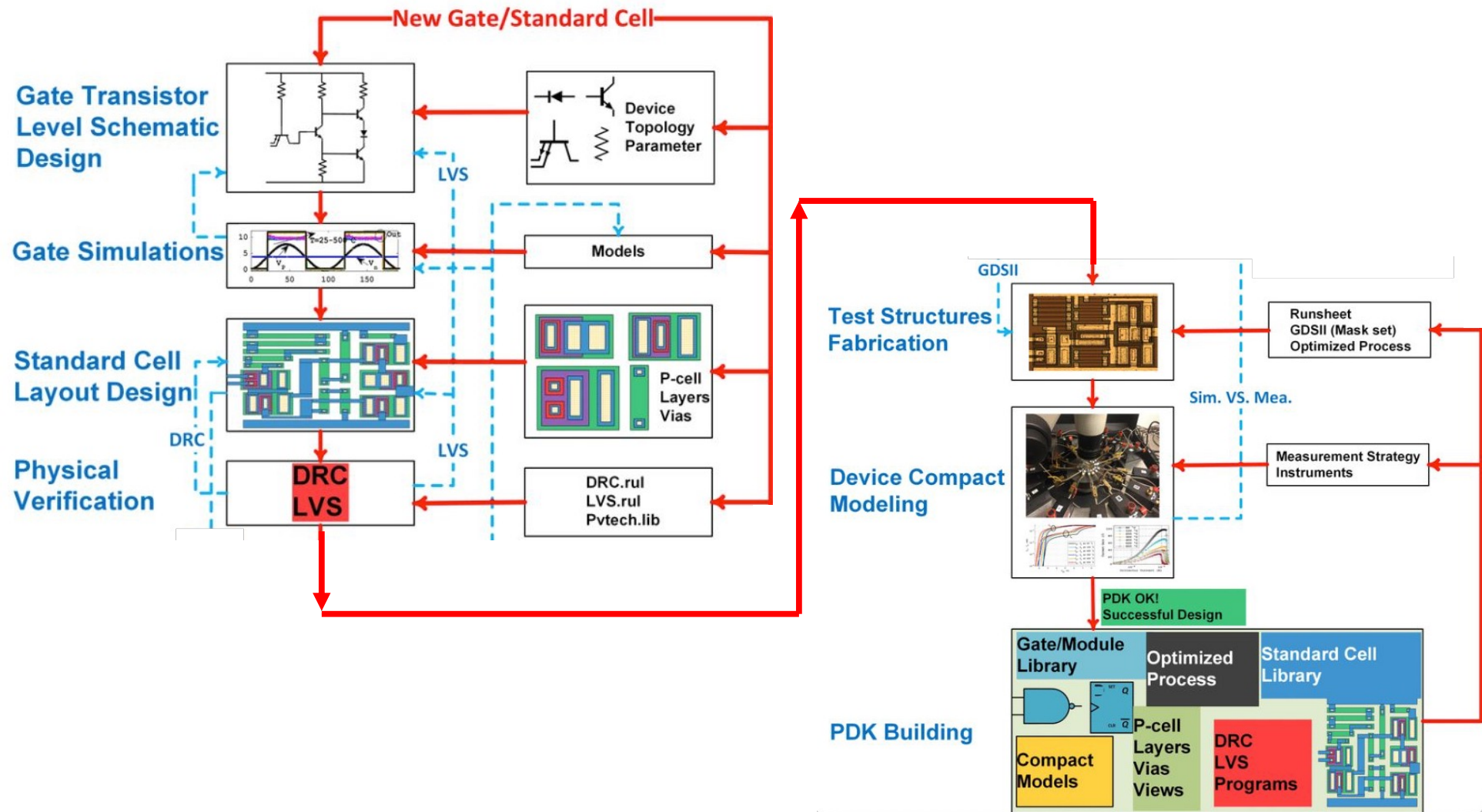


What is a Process Design Kit (PDK)

- ◆ Link between chip maker (foundry) and EDA tool supplier (TSMC → Synopsys)
- ◆ PDK is for a specific process (we use TSMC 65nm low power process)
- ◆ PDK consists for set of files in standardised format for different design steps:
 - Primitive devices (e.g. transistors, capacitor etc.)
 - Device parameters
 - Device symbol
 - Technology related information
 - Layers and layer names
 - Appearance (colour)
 - Constraints (e.g. current density)
 - Verification information
 - Design rule checking (DRC)
 - Layout vs Schematic (LVS)
 - Device extraction
 - Simulation models
 - Standard cell libraries
 - Documentation

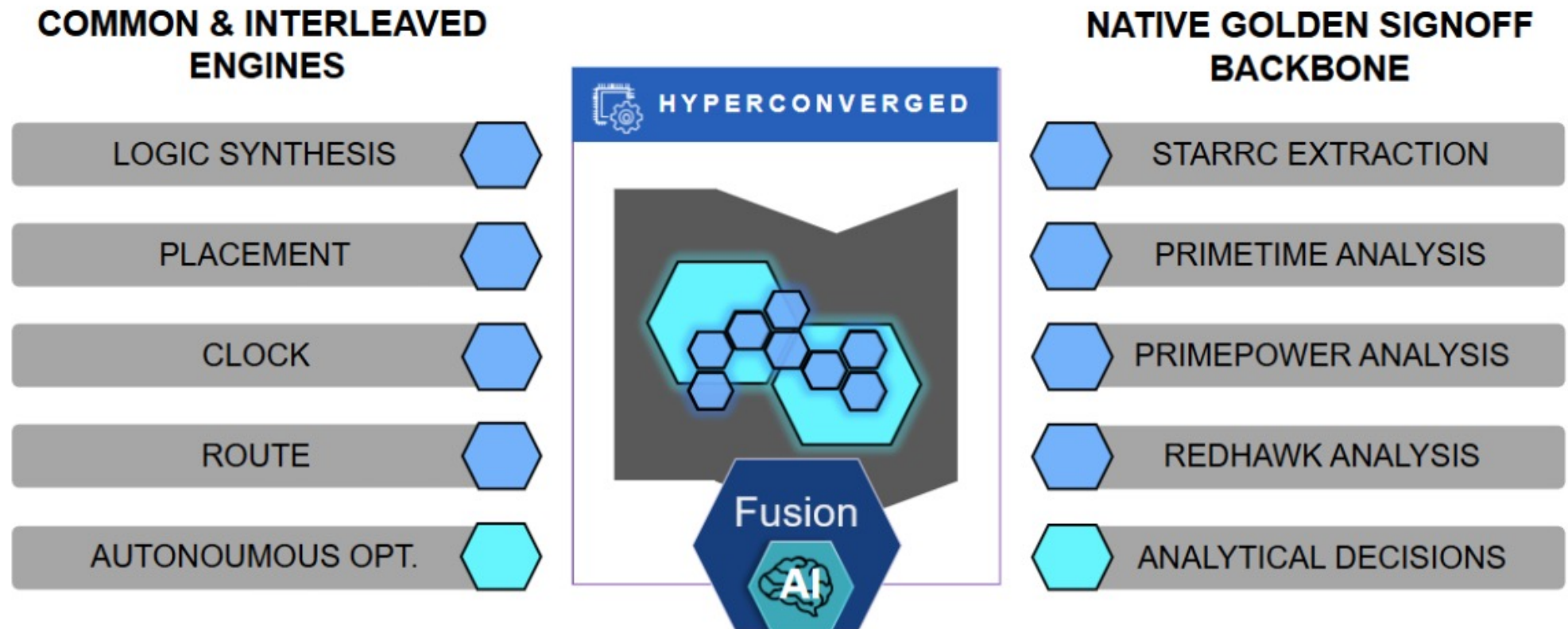


PDK Build Process



Source: Gupta

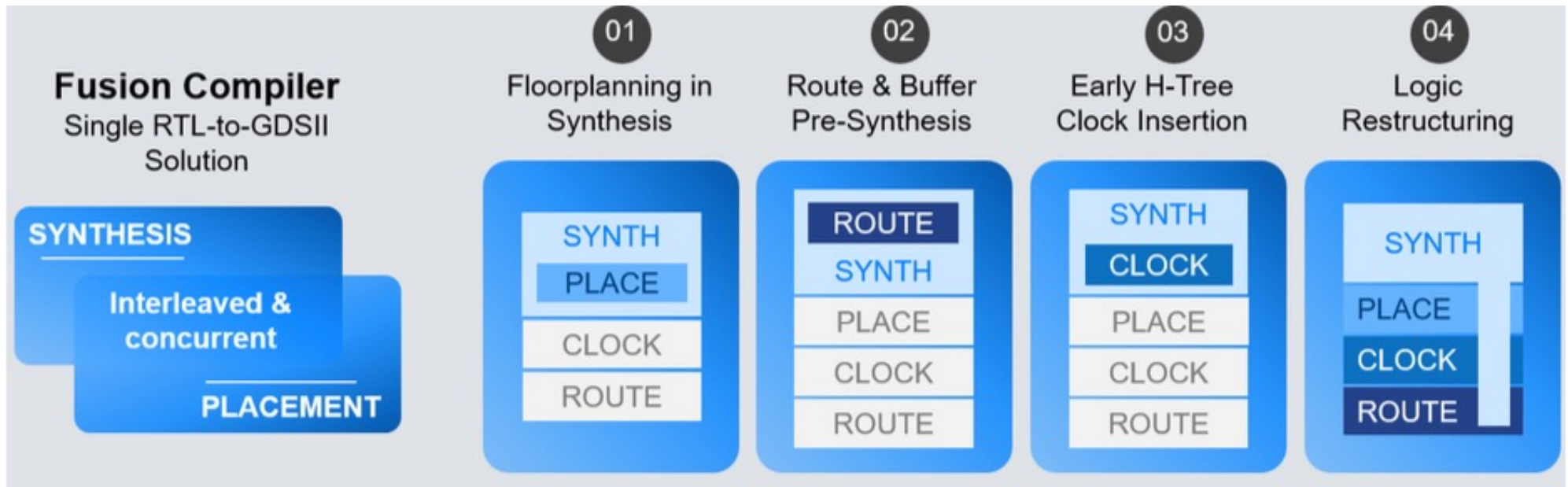
Synopsys Fusion Compiler



- ◆ Unified data model operating in a coherent environment.
- ◆ Avoid translation error.
- ◆ Constraints are consistent throughout.
- ◆ Speed up the learning curve of engineers.

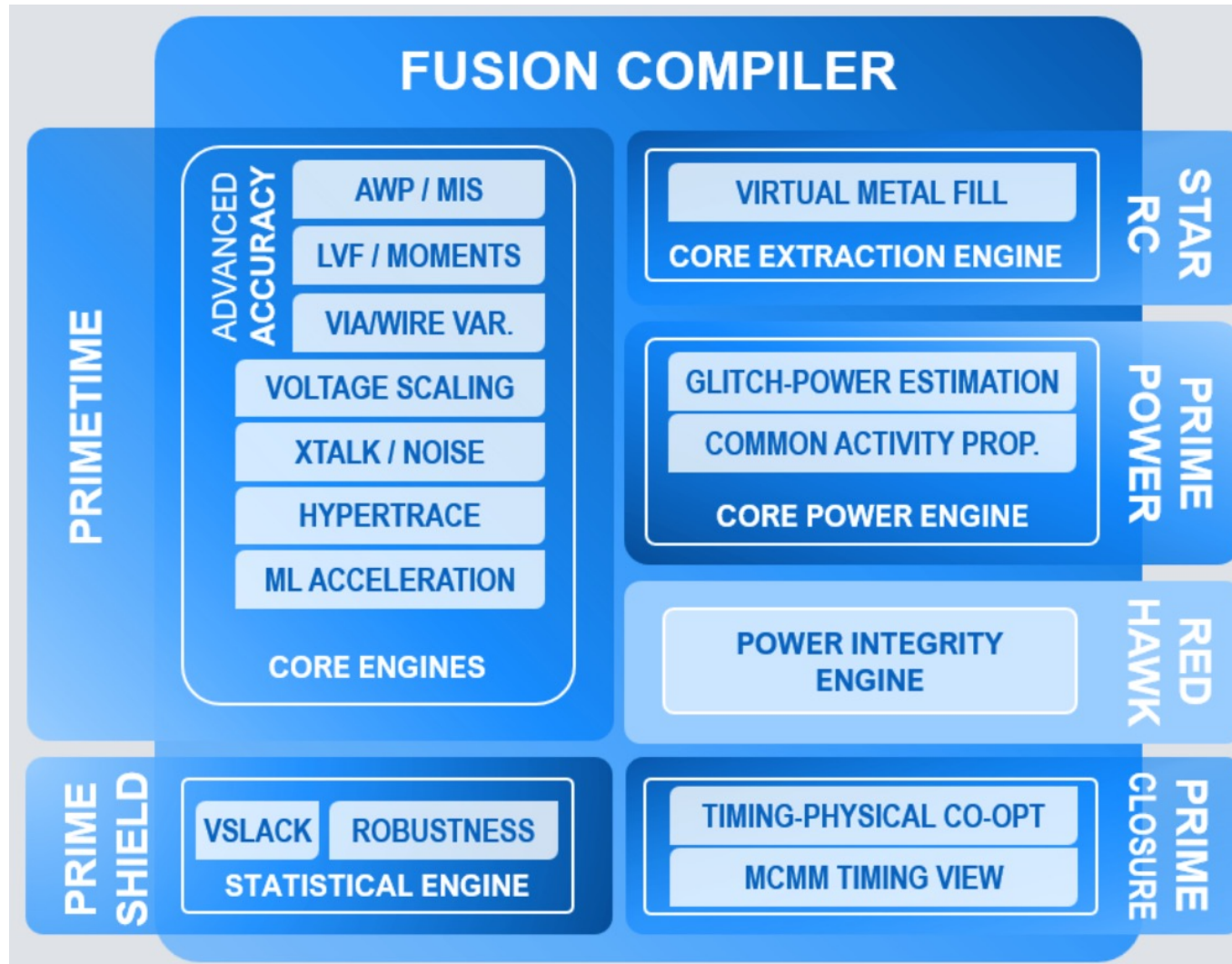
source: Fusion Compiler User Guide

Fusion Compiler Shell Engines



source: Fusion Compiler User Guide

Power Performance Area Optimization

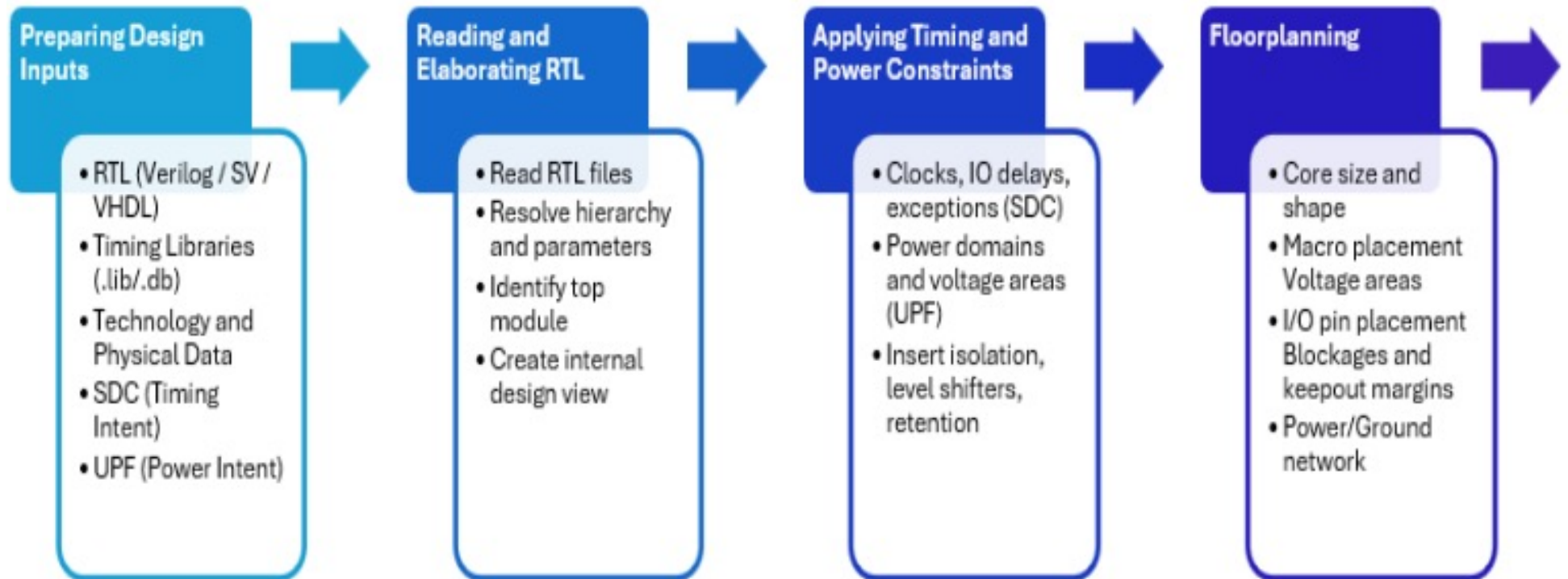


Trade off between:

- ◆ Power
- ◆ Performance
- ◆ Area
- ◆ FC iterates between these to resolve conflicts

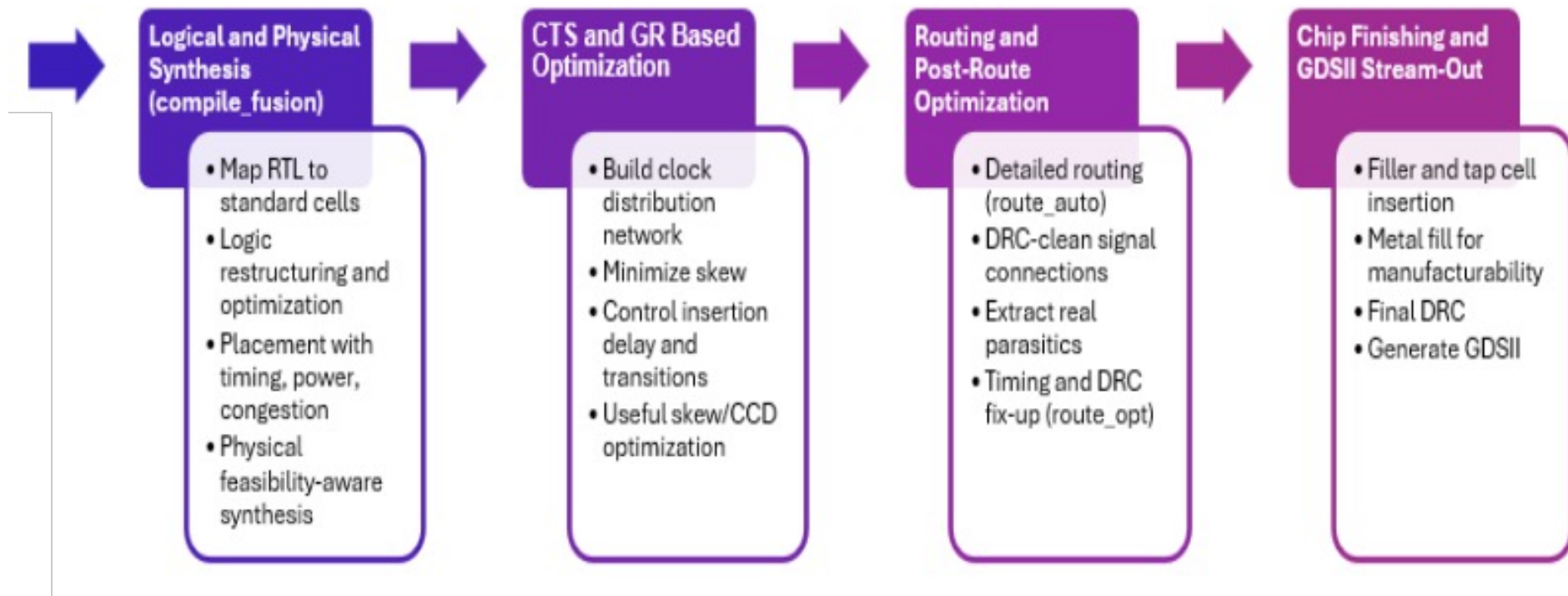
source: Fusion Compiler User Guide

RTL to GDSII Design Flow in FC (1 of 2)



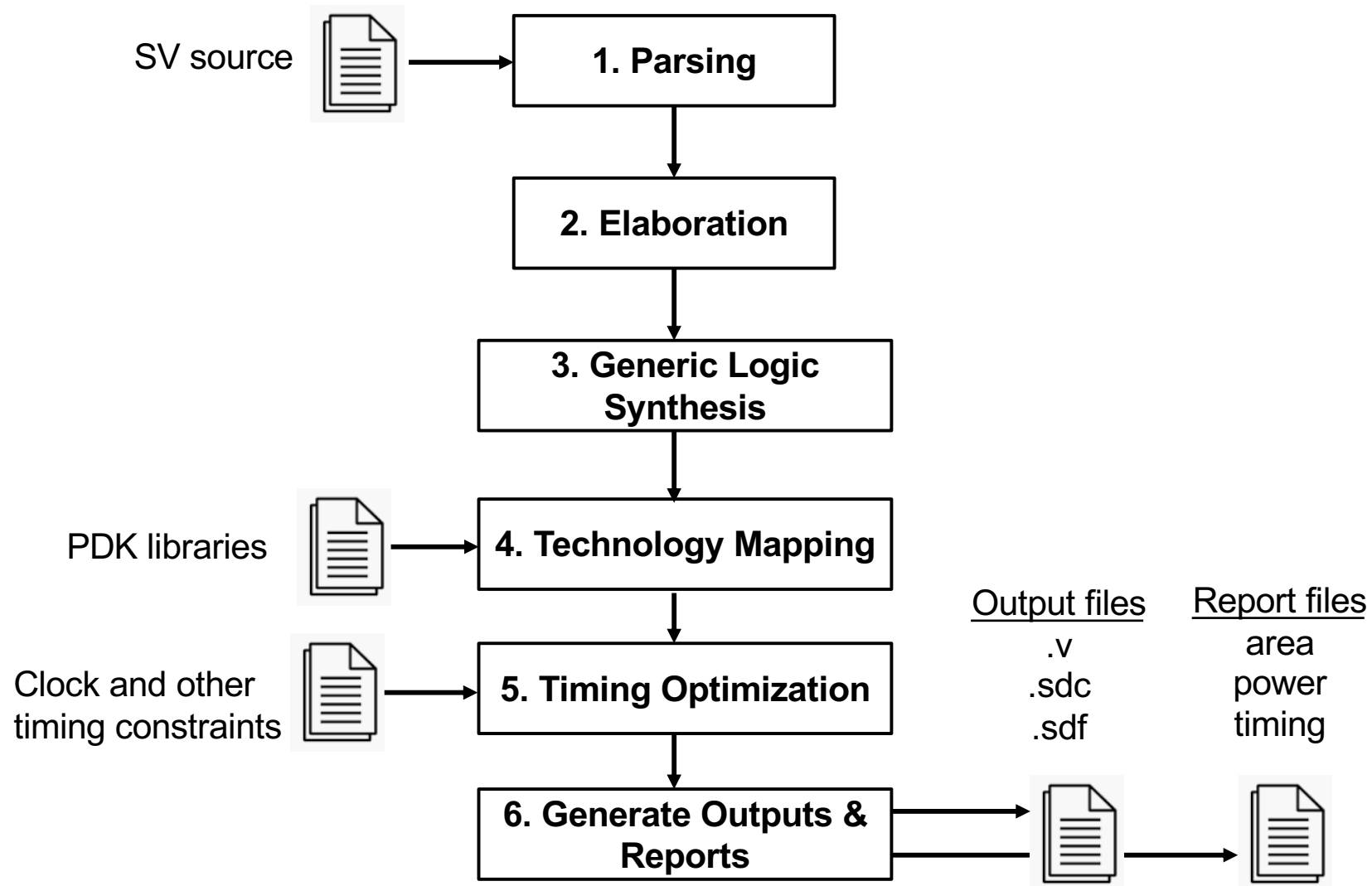
source: Fusion Compiler User Guide

RTL to GDSII Design Flow in FC (2 of 2)



source: Fusion Compiler User Guide

Lab 1 > Task 2: RTL to Standard Cell Synthesis



Synthesize LFSR4

```
module lfsr4 (  
    input logic      clk,      // clock  
    input logic      rst,      // reset  
    output logic [3:0] data_out // pseud  
);  
  
logic [4:1]      sreg;  
  
always_ff @ (posedge clk, posedge rst)  
    if (rst)  
        sreg <= 4'b1;  
    else
```

RTL source .sv

```
module lfsr4(clk, rst, data_out);  
    .....  
    wire n_0, n_1;  
    DFSNQD1BWP7T \sreg_reg[1] (.SDN (n_0), .CP (clk  
        | (data_out[0]));  
    CKXOR2D0BWP7T g16__8780(.A1 (data_out[2]), .A2  
        | (n_1));  
    DFCNQD1BWP7T \sreg_reg[4] (.CDN (n_0), .CP (clk  
        | .Q (data_out[3]));  
    DFCNQD1BWP7T \sreg_reg[3] (.CDN (n_0), .CP (clk  
        | .Q (data_out[2]));  
    DFCNQD1BWP7T \sreg_reg[2] (.CDN (n_0), .CP (clk  
        | .Q (data_out[1]));  
    INV0BWP7T g20(.I (rst), .ZN (n_0));  
endmodule
```

Synthesized circuit .v

Fusion Compiler
syn_logical.tcl

```
#####  
## Design  
#####  
  
set DESIGN    lfsr4  
set CLK_PORT  clk  
  
set RTL_FILES src/$DESIGN.sv  
set SDC_FILE  constraints/$DESIGN.sdc  
  
#####  
## Settings you may want to change  
#####  
# The clock is not here. It lives in t  
  
set CORE_UTIL    0.6      ;# fractio  
set ASPECT       1.0      ;# core he  
set CORE_OFFSET  5        ;# core to  
  
Tcl commands/script
```

```
-----  
data required time           0.78  
data arrival time           -0.00  
-----  
slack (MET)                  0.78
```

Timing report

Synthesis Steps (*setup.tcl*) (1)

```
#####
## Settings you may want to change
#####
## Design
#####
# The clock is not here. It lives i

set DESIGN    lfsr4
set CLK_PORT  clk

set RTL_FILES src/$DESIGN
set SDC_FILE  constrain

#####
set CORE_UTIL    0.6
set ASPECT       1.0
set CORE_OFFSET  5

set RING_WIDTH    1.0
set RING_SPACING  0.5
# Centres the ring set
# (CORE_OFFSET - (2 *
set RING_OFFSET  1.25

# Power rails on the 1
set RAIL_LAYER M1

# The cell rails are o
# rather than a crossi
set RING_H_LAYER M3
set RING_V_LAYER M2

#####
## Directories and files
#####

set WORK_DIR work
set OUT_DIR  outputs/$FLOW
set RPT_DIR  reports/$FLOW

set SYN_LIB $WORK_DIR/${DESIGN}_syn.dlib
set PNR_LIB $WORK_DIR/${DESIGN}_${FLOW}.dlib

set SYNTH_V    $OUT_DIR/${DESIGN}_synth.v
set SYNTH_SDC  $OUT_DIR/${DESIGN}_synth.sdc
set LAYOUT_V   $OUT_DIR/${DESIGN}_layout.v
```

Synthesis Steps (*syn_logical.tcl*) (2)

◆ Key inputs required for logic synthesis:

1. RTL source code (.sv)
2. Timing libraries (.lib or .db)
3. Technology files (.tf), parasitic models (.tlup) and physical libraries (.ndm)
4. Timing constraints (.sdc)
5. Power constraints (.upf) – not used in Lab 1

◆ The key steps in performing logic synthesis are:

1. Read and parse RTL
2. Elaborate the RTL design, check all hierarchy, link to library cells and submodules
3. Apply timing (and power) constraints
4. Perform logic synthesis (command: `compile_logical`)
5. Write report and export the design

Output of logic synthesis step

```
module lfsr4 (
    input logic      clk,          // clock
    input logic      rst,          // reset
    output logic [3:0] data_out    // pseud
);

logic [4:1]      sreg;

always_ff @ (posedge clk, posedge rst)
    if (rst)
        sreg <= 4'b1;
    else
        sreg <= {sreg[3:1], sreg[3] ^ sreg[4]};

assign data_out = sreg;
endmodule
```

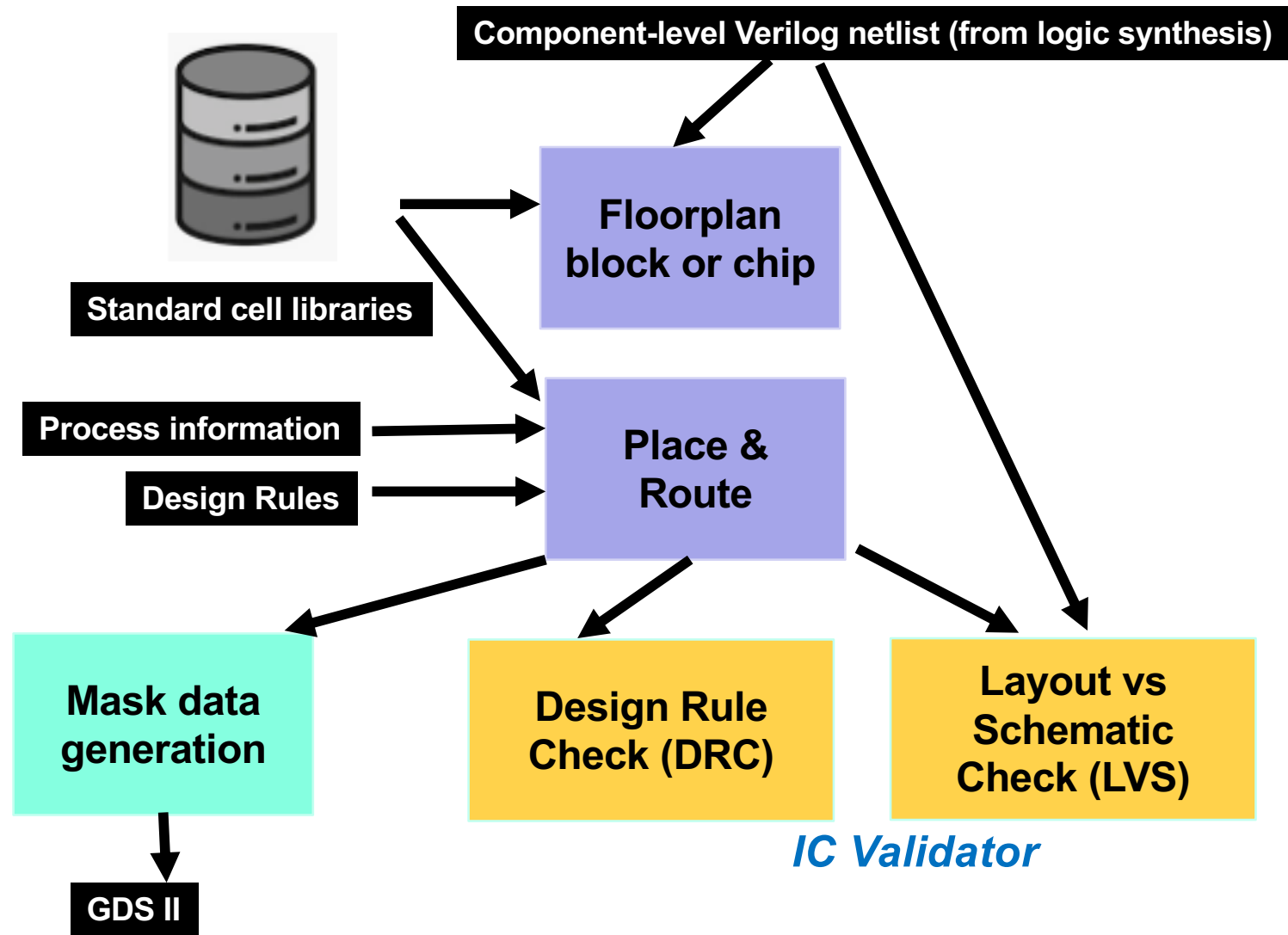
**RTL source
In SystemVerilog**

```
module lfsr4 ( clk , rst , data_out ) ;
input clk ;
input rst ;
output [3:0] data_out ;

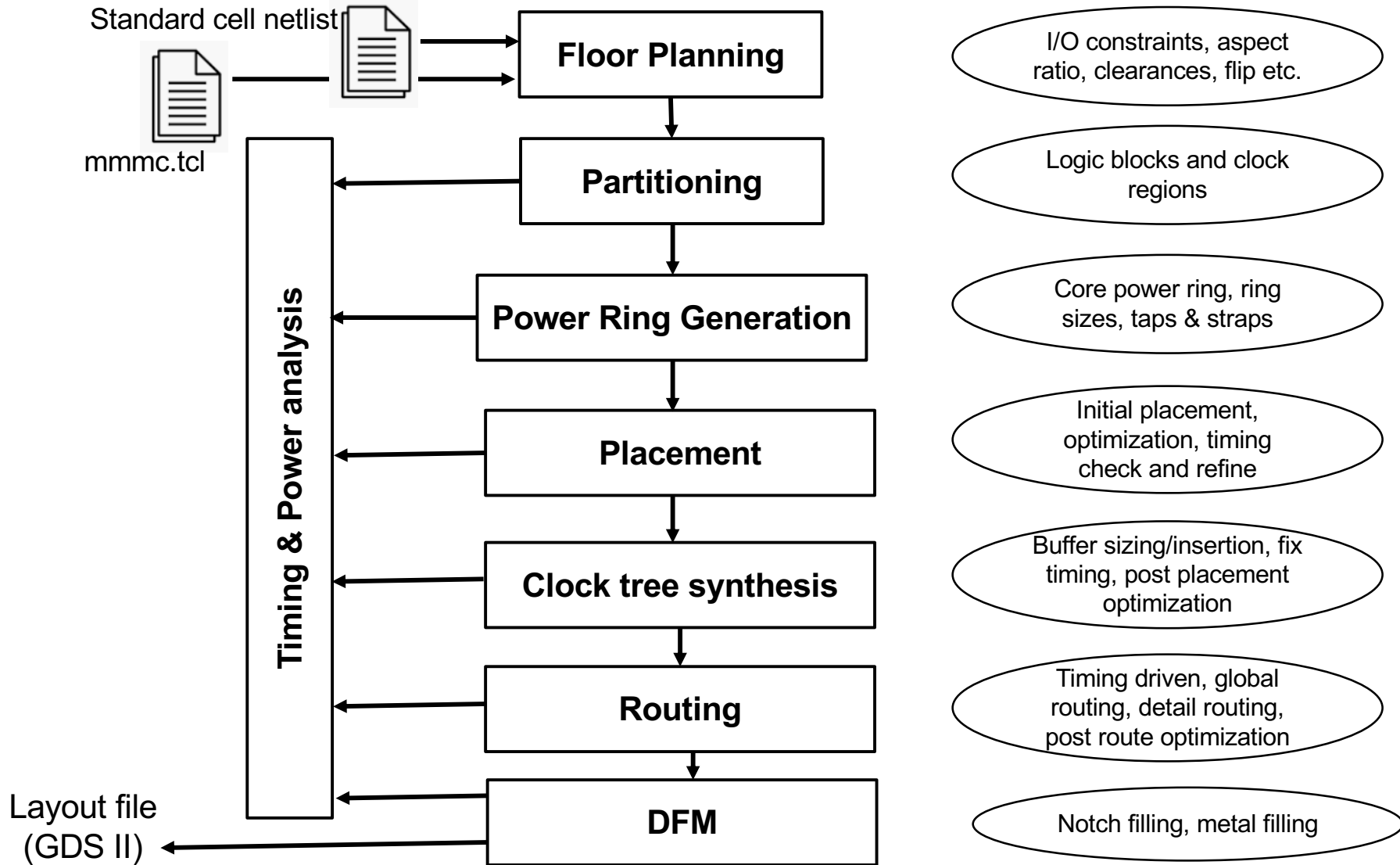
\*SEQGEN.1839103_120_2 \sreg_reg[4] ( .clear ( rst ) , .preset ( 1'b0 ) ,
    .next_state ( data_out[2] ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[3] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
\*SEQGEN.1839103_120_2 \sreg_reg[3] ( .clear ( rst ) , .preset ( 1'b0 ) ,
    .next_state ( data_out[1] ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[2] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
\*SEQGEN.1839103_120_2 \sreg_reg[2] ( .clear ( rst ) , .preset ( 1'b0 ) ,
    .next_state ( data_out[0] ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[1] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
\*SEQGEN.1839103_210_2 \sreg_reg[1] ( .clear ( 1'b0 ) , .preset ( rst ) ,
    .next_state ( N0 ) , .clocked_on ( clk ) , .data_in ( 1'b0 ) ,
    .enable ( 1'b0 ) , .Q ( data_out[0] ) , .synch_clear ( 1'b0 ) ,
    .synch_preset ( 1'b0 ) , .synch_toggle ( 1'b0 ) , .synch_enable ( 1'b1 ) ,
    .scan_enable ( 1'b0 ) , .scan_in ( 1'b0 ) ) ;
WVGTECH_XOR2 C20 ( .I0 ( data_out[2] ) , .I1 ( data_out[3] ) , .Z ( N0 ) ) ;
endmodule
```

**synthesized netlist
in Verilog (before tech map)**

FC Placement and Routing in context

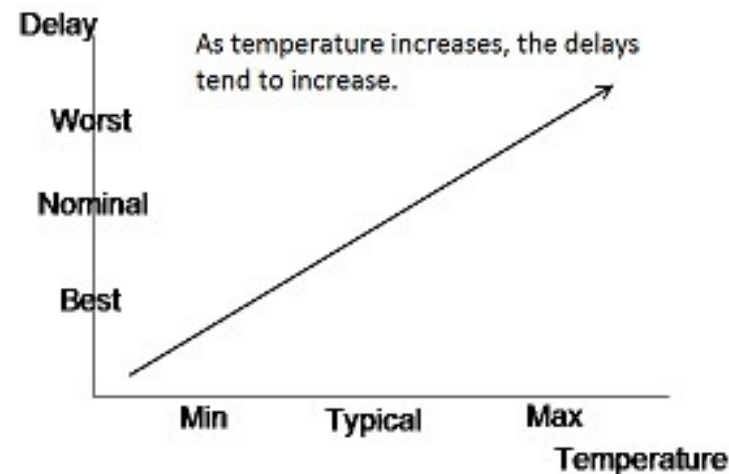
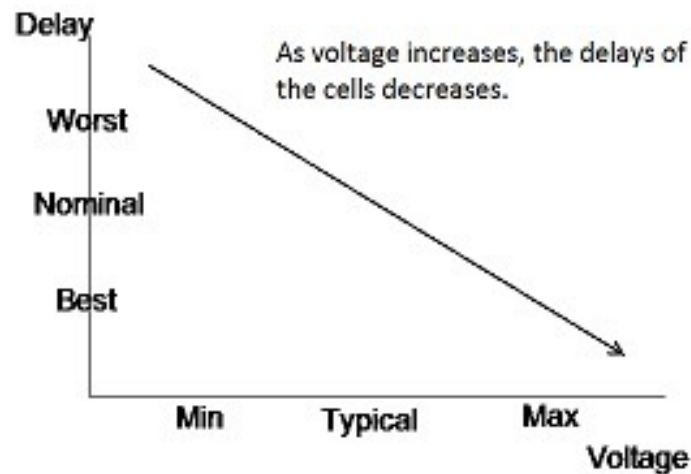
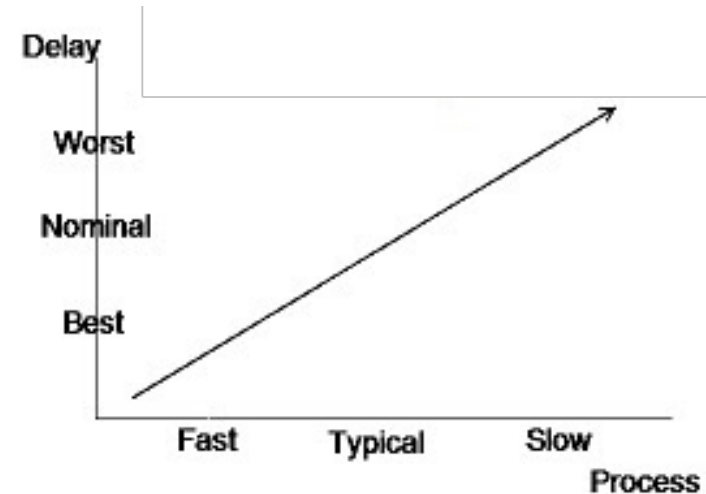


Netlist to Layout Flow



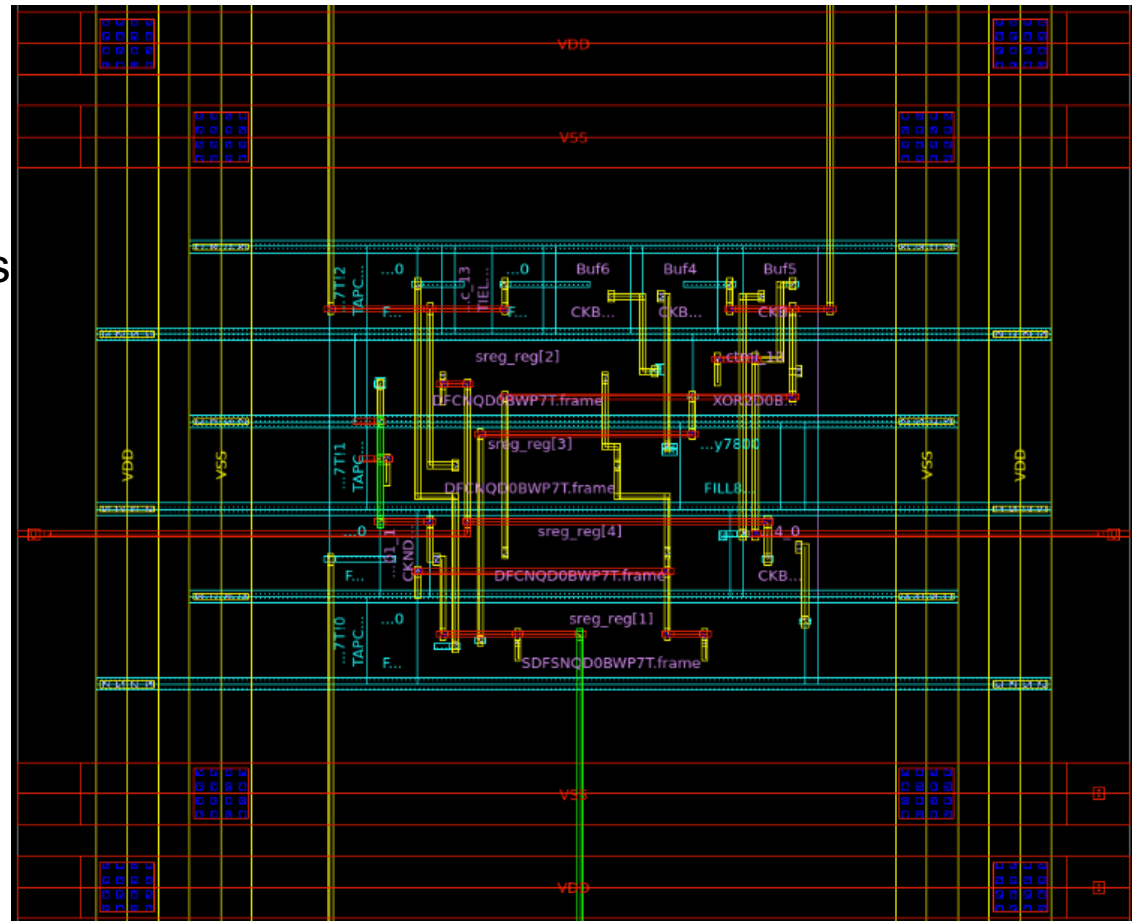
What is PVT?

- ◆ PVT stands for Process, Voltage and Temperature
- ◆ Need to tell Place-and-Route tools these variations
- ◆ Ensure circuit works when manufactured
- ◆ Must also work under worst case environment
- ◆ Specified PVT through MMMC file (Tcl format)



Results from PnR in Lab 1 Task 3

- ◆ To produce physical layout of the LFSR circuit, we need to:
 1. Perform floorplanning
 2. Perform placement
 3. Synthesize the clock tree
 4. Perform routing
 5. Check the PnR results for errors
 6. Export reports and outputs



Simulation using VCS

Testbench (lfsr4_tb.sv)

```
`timescale 1ns/1ps
module lfsr4_tb;
  logic rst;
  logic clk;
  logic [3:0] data_out;

  // instantiate the DUT
  lfsr4 dut (
    .clk(clk),
    .rst(rst),
    .data_out(data_out)
  );

  initial begin
    $display("Start lfsr4 testbench ....");
    clk = 1'b0;
    rst = 1'b1;
    #1 clk = 1'b1;
    #1 clk = 1'b0;
    #1 rst = 1'b0;

    // clock generator
    for (int i=0; i<20; i++) begin
      #1 clk = 1'b1;
      #1 clk = 1'b0;
    end
  end

  initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(1, lfsr4_tb, dut);
  end
endmodule
```

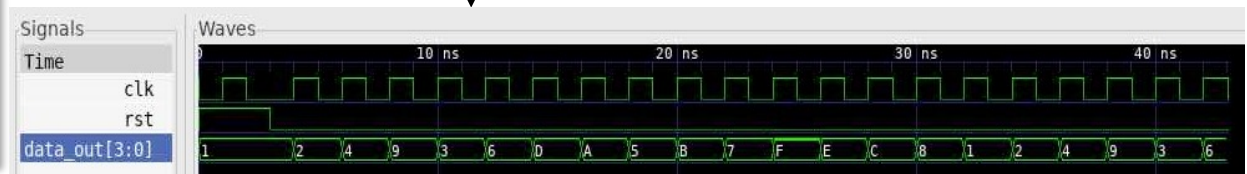
```
## sim-synth: simulate the synthesised netlist, real cells, no wire delay
sim-synth:
  @test "$(FLOW)" = logical || { \
    echo "sim-synth is for FLOW=logical. The fusion flow writes no"; \
    echo "intermediate netlist: placement carries straight through."; \
    exit 1; }
  @test -f $(OUT)/$(DESIGN)_synth.v || \
  { echo "No $(OUT)/$(DESIGN)_synth.v. Run scripts/syn_logical.tcl."; exit 1; }
  $(call run_sim,$(VCS) $$SYN_SIM_MODELS $(OUT)/$(DESIGN)_synth.v $(TB),synth)
```

Makefile

VCS simulator

waveform.vcd

GTKwave



Fusion Compiler's Mega Commands



source: Fusion Compiler User Guide

Further Reading

- ◆ Publicly available standard cell library:

https://www.vlsitechnology.org/html/cells/vsclib013/lib_gif_index.html

- ◆ Synopsys Fusion Compiler User Guide:

https://spdocs.synopsys.com/dow_retrieve/latest/dg/fcolh/pdf/fcug.pdf

- ◆ Fusion Compiler Landing webpage in SolvNet:

https://spdocs.synopsys.com/dow_retrieve/latest/home_public/fc.html

- ◆ Synopsys training page in SolvNet:

<https://training.synopsys.com/learn/course/external/view/elearning/529/fusion-compiler-synthesis-and-design-implementation-jumpstart>

- ◆ “New VLSI Tools”, Gajski&Kuhn ([10.1109/MC.1983.1654264](https://www.coursera.org/learn/new-vlsi-tools))

Access to Documents – SolvNet Registration

SolvNetPlus Account Registration

- Visit SolvNetPlus:
 - <https://solvnetplus.synopsys.com/>
- Click on:
 - "Register – Create Account"
- Fill out the required information
 - University email addresses and Site ID are required
 - Site ID for **Imperial College** is siteID 2727
 - Registration may take up to 24 hours to process

The image displays two screenshots of the Synopsys SolvNetPlus registration process. The left screenshot shows the 'Sign in' page with a 'REGISTER - CREATE ACCOUNT' link at the bottom. A red arrow points from this link to the right screenshot. The right screenshot shows the 'SolvNetPlus Self Registration' form, which includes fields for First Name, Last Name, Job Title, Email, Phone, Address, City, State/Province, and Zip/Postal Code. A 'Next' button is at the bottom right of the form.